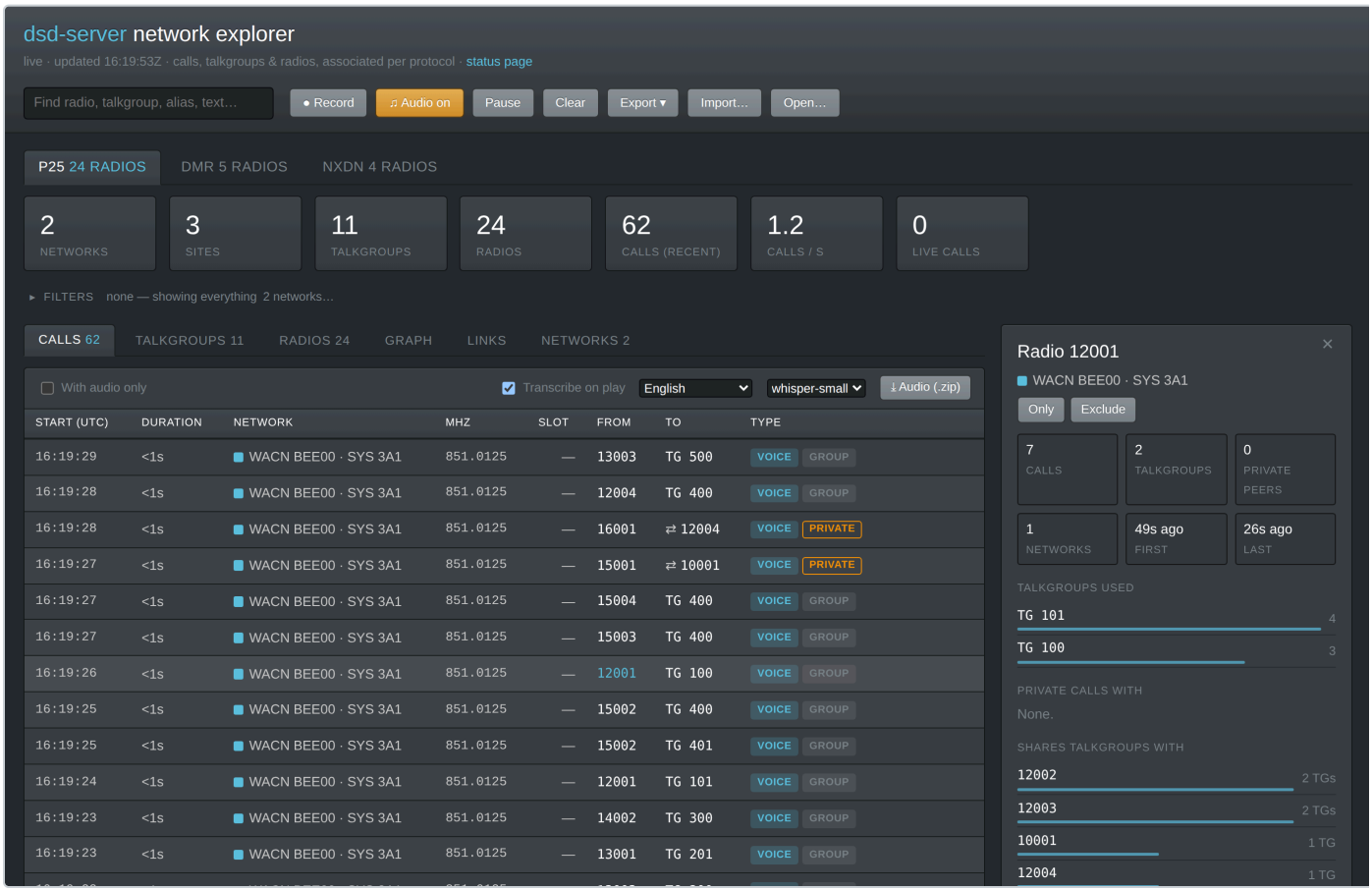
DSD-SERVER

# Network Explorer — User Manual

Using the network explorer: its views and controls, call audio and speech-to-text, saving and combining data, and how calls, talkgroups, radios and networks are associated for each protocol.

2026-10-16 · build 9a99cc4, 2026-10-16



The network explorer is the `/net` page of dsd-server. It turns everything the server decodes into **calls, talkgroups, radios and networks, and how they are connected**: who talks on which talkgroup, who calls whom privately, which talkgroups share radios, and which systems a radio or talkgroup appears on. It updates live, works on a PC, tablet or phone, and can save, reopen and combine its data.

This manual covers using the page. Appendix A explains how the associations are worked out, and Appendix B what that means for each protocol. Server settings, HTTP endpoints and file formats are in the README and PROTOCOL.md.

**Contents**


1. Getting started
2. The page at a glance
3. Calls
4. Talkgroups and Radios
5. Graph
6. Links
7. Networks
8. The details panel
9. Searching, filtering and sorting
10. Call audio
11. Speech-to-text
12. Saving, reopening and combining data
13. Recording a session for troubleshooting
14. Tablets and phones
15. Limits: what is kept, and for how long
16. Troubleshooting
17. Settings reference - Appendix A: How associations are built - Appendix B: Per-protocol notes - Appendix C: Glossary - Appendix D: The `calls.csv` columns


---

## 1. Getting started

1. Start dsd-server as usual and start one or more decode sessions from your client (DMR, P25, NXDN, dPMR, D-STAR, YSF, TETRA, EDACS / ProVoice, X2-TDMA). Paging (POCSAG / FLEX) is not shown in the explorer.
2. Open `http://<server>:<port>/net` in a browser — the same address and port as the status page (which links to it).
3. As traffic is decoded, a tab appears for each protocol and the views fill in. A channel with nothing on it, noise, or the wrong protocol selected leaves no trace.

**Send the channel frequency.** If your client's `start` message includes `center_freq` (the tuner centre; the channel is `center_freq + freq_offset`), the explorer labels and groups networks by channel (`Color Code 1 · 434.4250 MHz`) instead of by session (`Color Code 1 · stream 3`). That is much easier to read, and it lets

reconnects, restarts and a second receiver on the same channel join up. See Appendix A.2.

Nothing needs to be installed in the browser. The page is self-contained; only the optional speech-to-text loads extra files, and only when used.

The screenshots in this manual use demo data (a simulated P25 system, NXDN and DMR sites, and one DMR test call); transcripts in them are illustrative.

## 2. The page at a glance

From top to bottom:

| Part | What it does |
|---|---|
| **Status line** | `live · updated 12:34:56Z`, or `paused`, `file view`, `disconnected — retrying`. Links back to the status page. The page updates every 1.5 s, but waits while you are in the middle of something — `live · held: clicking` (a button or mouse held down, a drag), `held: scrolling`, or `held: text selected` (so you can copy an id; click anywhere to clear the selection) — and catches up right after. Where the details panel was scrolled to stays put across updates. |
| **Search box** | Finds radios, talkgroups, aliases, SMS text and frequencies. Filters every view. See section 9. |
| **Record** | A developer tool, hidden unless turned on (section 13): records everything the explorer receives, for replay and troubleshooting. While on, the header shows the file name, its size and a **Download** link. |
| ♫ **Audio** | Starts / stops recording each call's voice (off by default) — section 10. Orange when on. |
| **Pause** | Freezes the whole page (no updates) until pressed again. |
| **Clear ▾** | A menu (asks first for each): **Learned data** forgets the calls, talkgroups, radios and networks learned so far and any imports — and, if there are recorded call audio files, offers to delete those from the server too (Cancel keeps them) — but keeps your keys and merges; **Network merges** unmerges every merged network, across all protocols; **Encryption keys** removes every stored key, across all protocols (cannot be undone). |
| **Export ▾** | Saves what the explorer shows: explorer data (.json), the same with the calls' audio (.zip), the association graph (.graphml), or the listed radios' positions (.kml). |
| **Import…** | Adds saved exports (e.g. from another receiver) to the live view. |
| **Open…** | Views saved exports offline, read-only (several files are merged). You can also drop files onto the page. |
| **?** (Help) | Opens this manual (PDF) in a new tab. It is built into the server, so it matches the server's version and needs no internet. |
| **Protocol tabs** | One per protocol with traffic (e.g. `DMR 62 radios · 1 live`), or with a client connected for it (even before any traffic). A dot marks a protocol with a decode stream connected: **green** while one is decoding (anything decoded in the last 10 s), **hollow** when connected but quiet. Hover for the streams. The busiest protocol comes first when the page loads; after that the tabs keep their places (a protocol that appears later goes at the end), so they don't swap while counts are close. Reload to re-sort. Protocols are never mixed: a DMR radio 1234 and a P25 radio 1234 are unrelated. |
| **Connections** | At the end of the tabs row: `● 3 clients · 2 streams` — clients connected to the server and the decode streams they run (green dot while any decodes), or `No clients connected`. Hover for each stream's protocol, frequency, and when it last decoded something. |

| Part | What it does |
|---|---|
| **Stat cards** | Networks, Sites, Talkgroups, Radios, Calls (recent), **Streams** (this protocol's connected decode streams, green while one decodes; hover for each; not in a file view) and Live calls — for the current protocol, network filter and search (not **With audio only** or **Hide signaling**, which only shorten the Calls list). The protocol tab's "N live" counts all of its networks, so with a network filter or a search the two can differ. |
| **Filters** | Everything that narrows the page, in one section under the stat cards (§9). Collapsed (the default) it is one row: ▸ **Filters**, a chip for each filter in force (click one to drop it) and **Clear all**; with none it says *none — showing everything*. Click ▸ **Filters** to open it: a row each for networks (one chip per network, with its call count), talkgroups and radios (each with a box to type ids into), and the search. The browser remembers whether it is open. |
| **View tabs** | Calls · Talkgroups · Radios · Graph · Map · Links · Networks (and **Changes** while comparing to a baseline). |
| **Details panel** | On the right (or a drawer on smaller screens): everything tied to the radio, talkgroup or network you click. |

On a narrow window the header's buttons fold into a ☰ **Menu**.

When the server is updated to a new version of the explorer, open pages reload themselves (no need to refresh): the protocol, filters, search and selection are kept. The reload waits while you are busy — audio playing, speech-to-text running, a click, a text selection or typing — and a note says it will happen when you're done. Restarting the server with the same version doesn't reload anything.

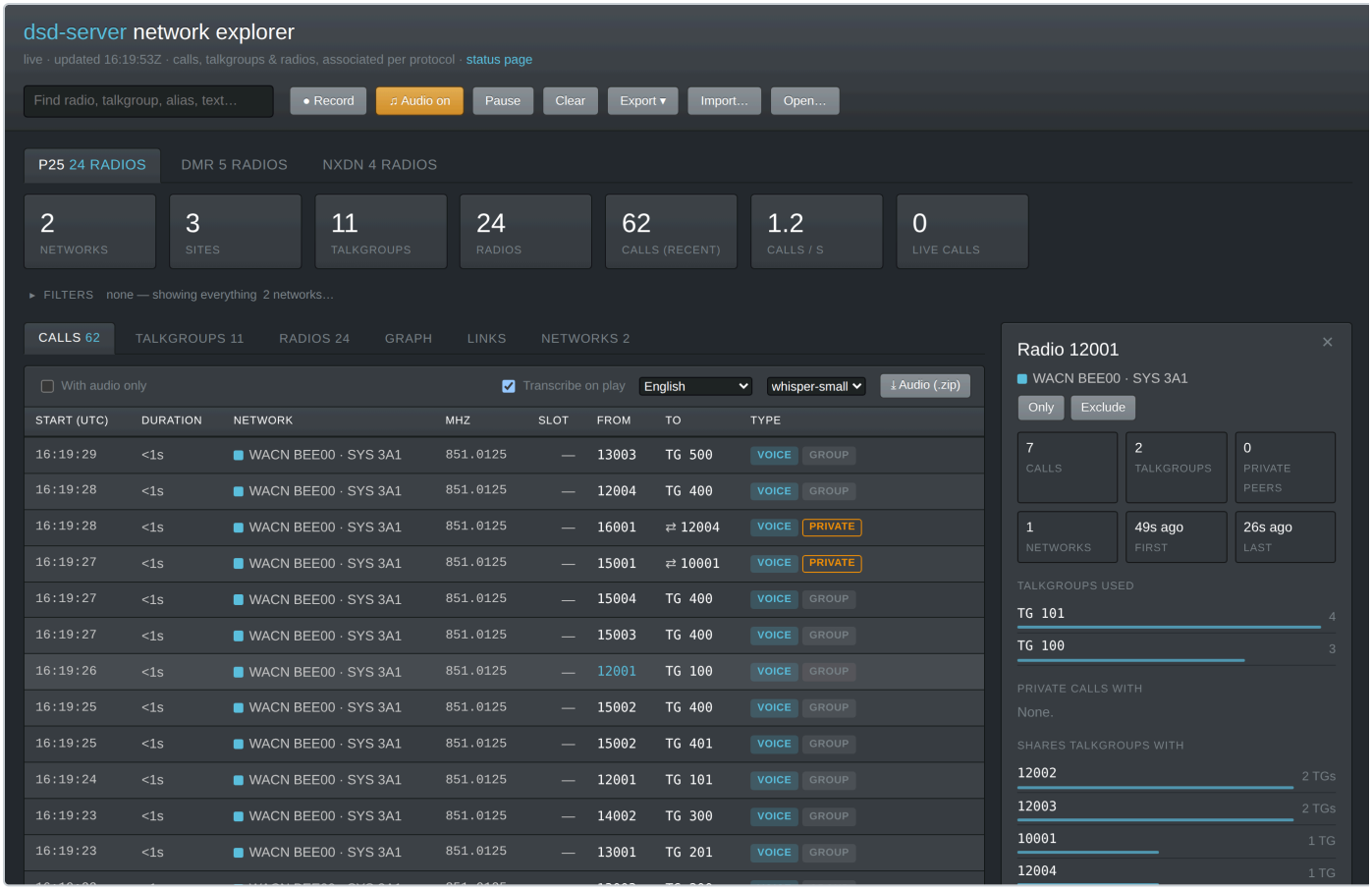



Figure 1: The explorer on a PC: header, protocol tabs, stat cards, the filters (collapsed), the Calls view and, on the right, the details of the radio clicked (12001), with ✕ to deselect it.

## 3. Calls

One row per call, newest first. A *call* is stitched together from many decoder lines (sync, voice frames, call headers, aliases, SMS) — see Appendix A.3.

| Column | Meaning |
|---|---|
| **Start (UTC)** | When the call began. |
| **Duration** | How long it lasted; a pulsing green dot and running time while it is live. A DMR call ends when the talker unkeys: the repeater's hang time after it (terminators repeating the call's ids for a few seconds) isn't counted, though a talker keying up again within it continues the same call. The audio can still be a little shorter: silence at the start of a transmission isn't recorded. |
| **Network** | The network it was heard on (coloured swatch). When the MHz column already shows the frequency, it is left out of the name here; hover for the full name. |
| **MHz** | The channel frequency (when the client sent it). |
| **Slot** | TDMA timeslot where the decoder reports one (DMR's 1 or 2; P25 Phase 2); `—` otherwise. |

| Column | Meaning |
|---|---|
| **From** | The talking radio, with its talker alias when decoded. Click it for the radio's details. |
| **To** | The talkgroup (`TG 1234`), or `⇄ 5678` for a private (radio-to-radio) call. Click it for details. |
| **Type** | Badges: **VOICE**, **DATA**, or **SIGNALING** (a call was announced — source / target decoded — but no voice or data was heard on this channel, so there is no audio; often a control-channel grant, a hang-time message, or noise); **GROUP** or **PRIVATE**; **EMERGENCY**; **ENCRYPTED** (hover for the algorithm and key id it named); and **2 RX** when two receivers heard the same call (it is listed once). Hover for an explanation. |
| **Audio** | ▶ **0:12** plays the call's recorded voice, ⭳ downloads it — only when audio recording is on (section 10). |
| **Content** | What the call carried: an SMS / text message, or — for a data call without text — what kind of data, in grey italics: **ACK (delivery confirmed)**, **Data packet**, **ARS (registration)**, **LRRP (location)**, **TMS (text message)**, **Moto data (service 0x…)** for a Motorola (MNIS) service with no published format (0x80 and 0x20 are common — binary packets from radios to the data gateway, kept undecoded rather than shown as nonsense text), or **Data announced only** when only the announcement was heard here (the data may have gone out on another channel). An encrypted call shows the algorithm and key id it announced: **AES-256 · key 0x666A**. Motorola ARS packets are decoded: **ARS registration · radio 21518** (a radio registering with the system's data gateway, by its device id) and **ARS registration ACK** (the gateway's answer). A position report sent during the call shows as 📍 **lat, lon**, linked to Google Maps. This column only appears when some call has any of these. |

A call's **transcript** (when speech-to-text is used) appears in quotes on its own line under the row.

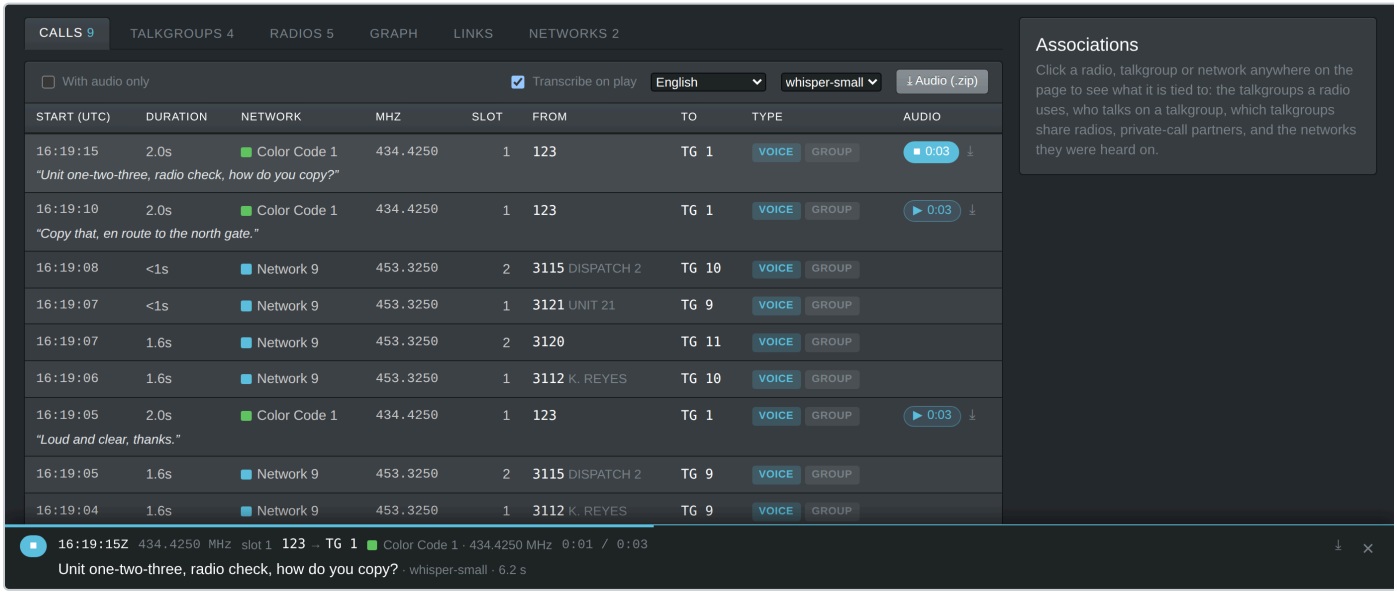



Figure 2: DMR calls with recorded audio (▶ and ⭳), transcripts under their calls, and the now-playing bar at the bottom of the window.

**Calls toolbar** (above the list):

- **With audio only** — lists only calls with recorded voice.
- **Hide signaling** — hides **SIGNALING** calls (announced, but no voice or data heard here). Appears only when there are any. Like **With audio only**, it only shortens the Calls list; the stat cards still count every call.
- **Transcribe on play**, the language and model lists — see section 11.

- ⭳ **Audio (.zip)** — downloads the audio of the calls listed (section 10).

The audio and transcription controls appear once audio recording is on or some call has audio.

Up to 400 calls are rendered at a time; the explorer keeps many more — narrow the list with the search box, the **Filters** or a sort to see older ones. It keeps up to 10000 calls **without** audio per protocol (`DSD_NET_MAX_CALLS`), and **every call that still has recorded audio on the server** on top of that — so a call you can play never silently drops off the list while its audio is kept. How much audio is kept is a disk limit, not a call count (`DSD_NET_AUDIO_MAX_MB` / `DSD_NET_AUDIO_MAX_AGE_H`, §10); when a call's audio is pruned to stay under it, that call then ages out by the 10000 count like any other.

## 4. Talkgroups and Radios

**Talkgroups**: one row per talkgroup — the networks it was heard on, how many radios used it, its call count, emergency and encrypted call counts, and when it was first and last heard.

**Radios**: one row per radio — its busiest talkgroups, how many talkgroups and private-call partners it has, its networks, call count and when it was last heard. Aliases are shown next to the id wherever the radio appears.

Click a row for its details (section 8). Click a column header to sort.

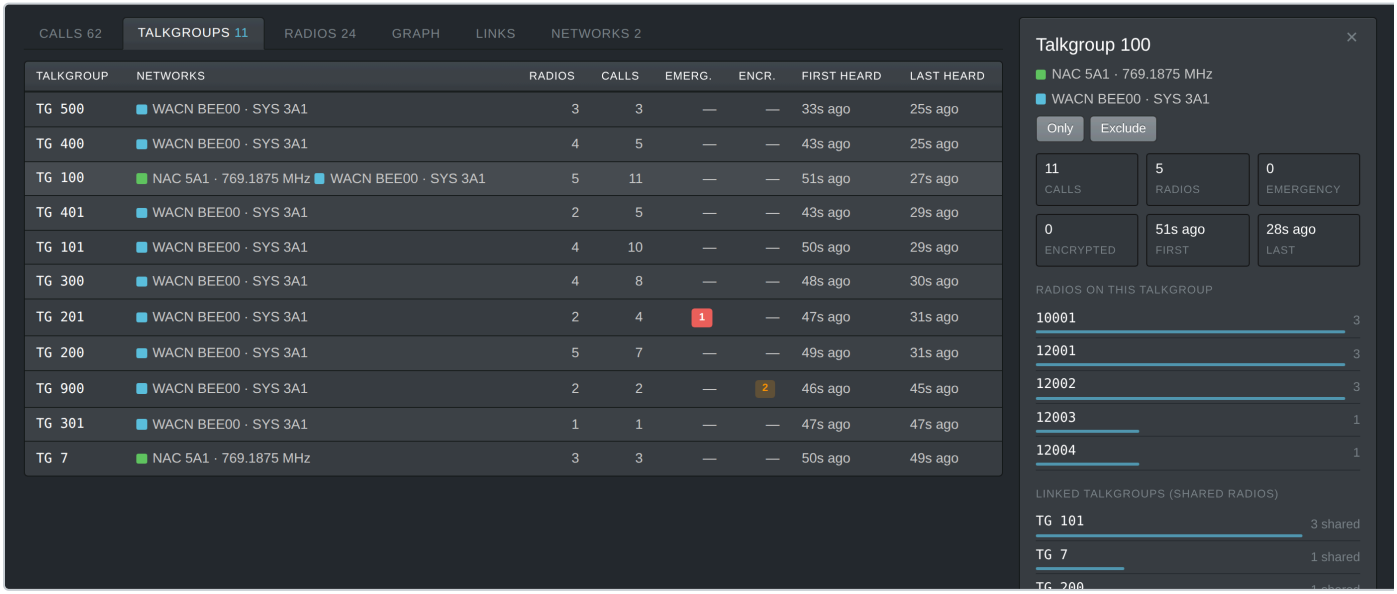



Figure 3: Talkgroups, with TG 100 selected: its radios and the talkgroups linked to it through shared radios.

## 5. Graph

A live picture of who talks on what: **talkgroups** and **radios** are nodes, a line joins a radio to each talkgroup it used, and a **dashed orange line** joins two radios that made a private call. Node colour is the network; a **white dashed ring** marks a radio or talkgroup seen on two or more networks (drawn in the colour of the busiest of

them when it first appears, and it keeps that colour while it stays on that network, rather than changing with every shift in traffic); a talkgroup's size grows with its calls.

The layout arranges itself: linked nodes pull together and every node pushes the others away, harder the more links it has. So a busy talkgroup, or a radio with many private-call partners (a data gateway, a dispatcher), keeps its own space with its radios around it, other talkgroups stay out of its ring, and nodes never sit on top of each other. Private-call links pull less than talkgroup links, so the talkgroups shape the picture. Groups that aren't connected to each other are pushed apart as whole clumps, so an unrelated cluster doesn't end up tangled inside a busy one.

- **Nodes** — how many of the busiest radios and talkgroups to draw (100 to 1000). The note on the right says how many are shown.
- **Private-call links** and **Radio labels** — show or hide them.
- Drag a node to move it; drag the background to pan; scroll, pinch or use **− / +** to zoom; **Fit** brings everything back into view. Until you zoom or pan yourself, the view keeps the whole graph in sight as it grows (it eases out to fit when a node would leave the frame); once you have zoomed or panned it stays where you put it, and **Fit** turns following back on.
- Click a node to select it: its neighbours stay lit, everything else fades, and the details panel shows it.

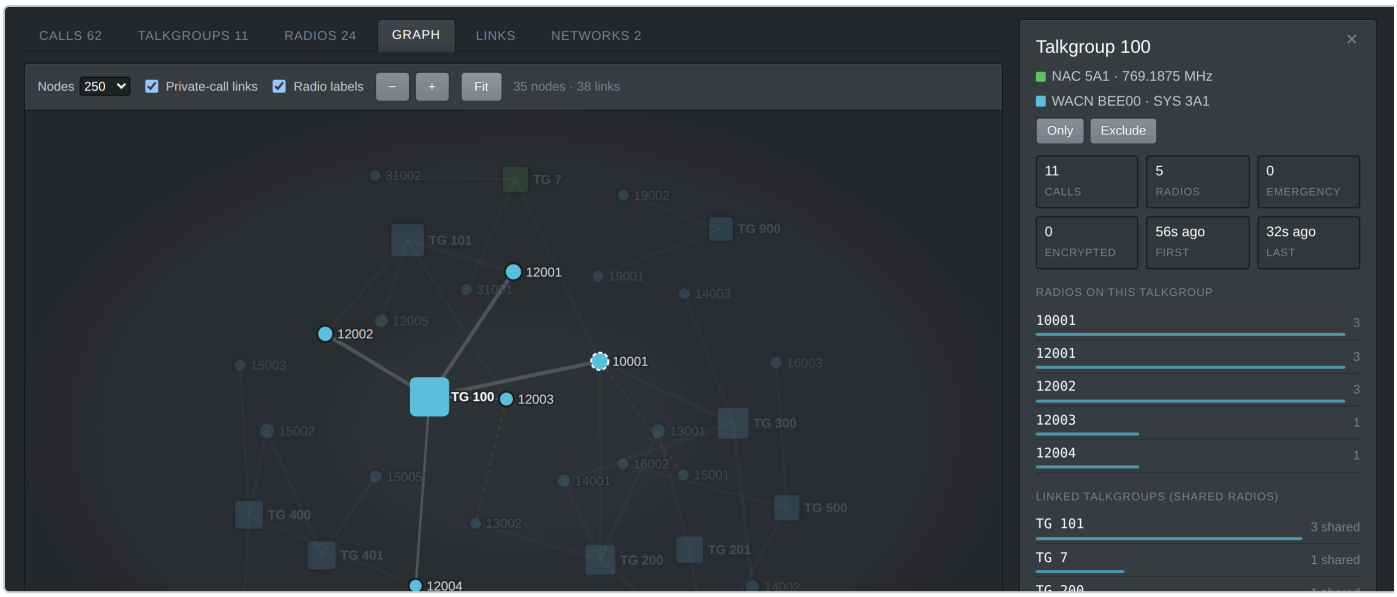



Figure 4: The graph with TG 100 selected. Squares are talkgroups, dots radios; the white dashed ring (10001) marks a radio seen on two networks.

## 6. Links

The analysis view — evidence of how radios, talkgroups and systems relate:

- **Probably the same network** — merge suggestions: networks that look like one channel heard at different frequency offsets, each with its reasons, **Merge** and **Not the same** (Merging networks). Shown first when there are any. A network with no decoded identity (**Unidentified · …** — only a frequency, nothing else) is never suggested: it has nothing to match on and comes and goes with its stream.

- **Talk communities** — groups of radios tied together through talkgroups they share or private calls between them, largest first. A community is often one agency, fleet or work group. Click a community's heading (**Filter ▸**) to pin it as a filter: the graph, the Calls, Talkgroups and Radios lists and the stat cards then show only that community's members, and a **Community · Nr · Mt** chip appears in the Filters row. Click the heading again (now **✓ filtering**), the chip's ✕, or **Clear all** to drop it.
- **Seen on more than one network** — talkgroups and radios heard on two or more networks. On weakly identified systems (see Appendix A.2) this is how you spot that two "networks" are really the same system, or that systems are linked.
- **Hub radios** — radios active on three or more talkgroups: dispatchers, supervisors, scanning or patched radios.

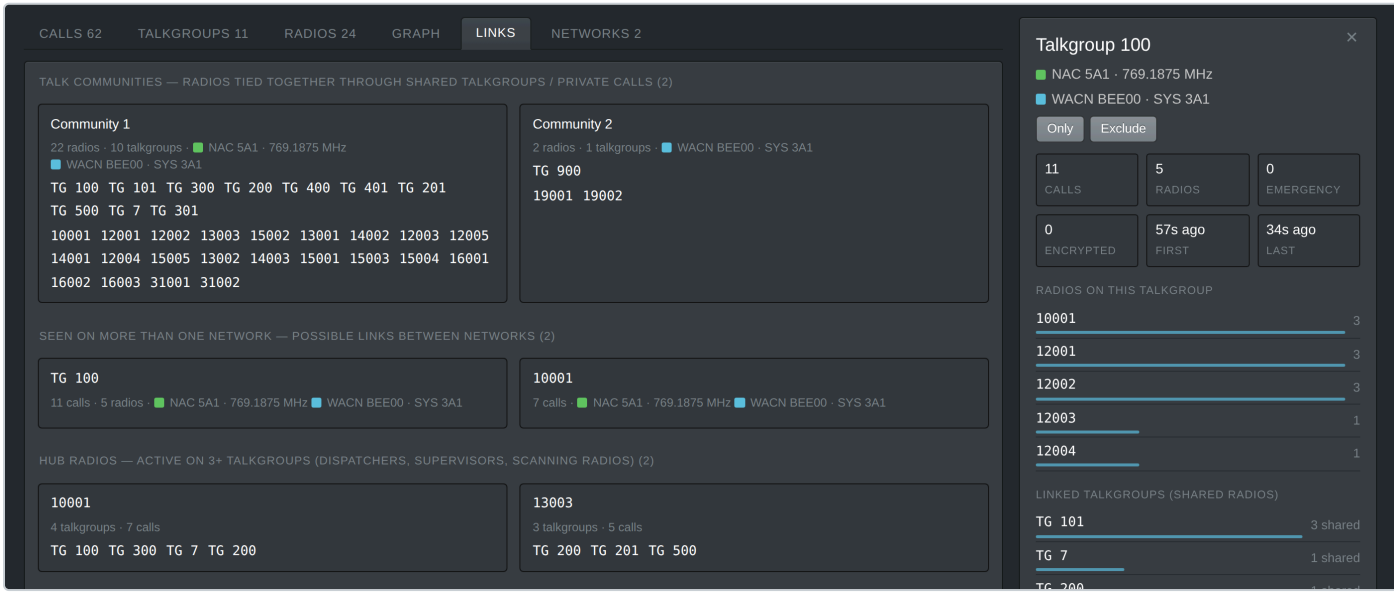



Figure 5: Links: talk communities, a talkgroup and a radio seen on two networks, and hub radios.

## The Map view

Radios that sent **position reports** (DMR LRRP / GPS, P25 Motorola GPS): each one's latest position as a labelled dot in its network's colour, and — with **Paths** ticked — the path through its positions, in time order and straight from fix to fix (no road snapping). A path starts at a small ring (its first fix) and ends at the radio's labelled dot (its latest). Point at a path or a dot to pick out that radio — its path, ring and dot stay bright, everything else fades — and click either to select the radio: its path stands out, with every fix, and the others stay faded until you deselect it. Drag to pan, scroll or **+ / −** to zoom, **Fit** to see them all. It follows the filters and the search.

The map is drawn by the page from **map tiles** — no key or account. **Map** picks the tiles: **OpenStreetMap**, **CARTO Dark**, **Esri imagery** (satellite), **OpenTopoMap**, or **No map** (positions only, and the sketch in a radio's details); the browser remembers the choice. The browser fetches the tiles itself, so it needs the internet — or set `DSD_NET_MAP_TILES` to a tile server of your own (e.g. a local OpenStreetMap server, for use offline); it then comes first as **This server's map**. A single position (📍 in a call, a radio's fixes) opens in **Google Maps**.

# 7. Networks

One row per network the explorer has identified:

| Column | Meaning |
|---|---|
| **Network** | Its name: a system id (`WACN BEE00 · SYS 19E`, `Network 12`, `System 4A1`, `MCC 234 · MNC 14`), or a short code with a channel (`Color Code 5 · 460.0250 MHz`) or a stream (`Color Code 1 · stream 3`), or `Unidentified · …`. |
| **Identity** | How sure the identification is: **strong**, **channel**, **weak** or **unidentified** (hover for the meaning; Appendix A.2). |
| **Identifiers** | The raw ids decoded (`cc=5 site_id=2 network_type=cap+` …). |
| **Sites** | Sites reported (`RFSS 1 · Site 3`, `Site 2`, `LA 1001` …), including neighbour sites announced by the system. |
| **Channels (MHz)** | Frequencies it was heard on. |
| **TGs / Radios / Calls** | Counts on this network. |
| **Streams** | How many decode sessions fed it (more than one = several receivers or reconnects). |

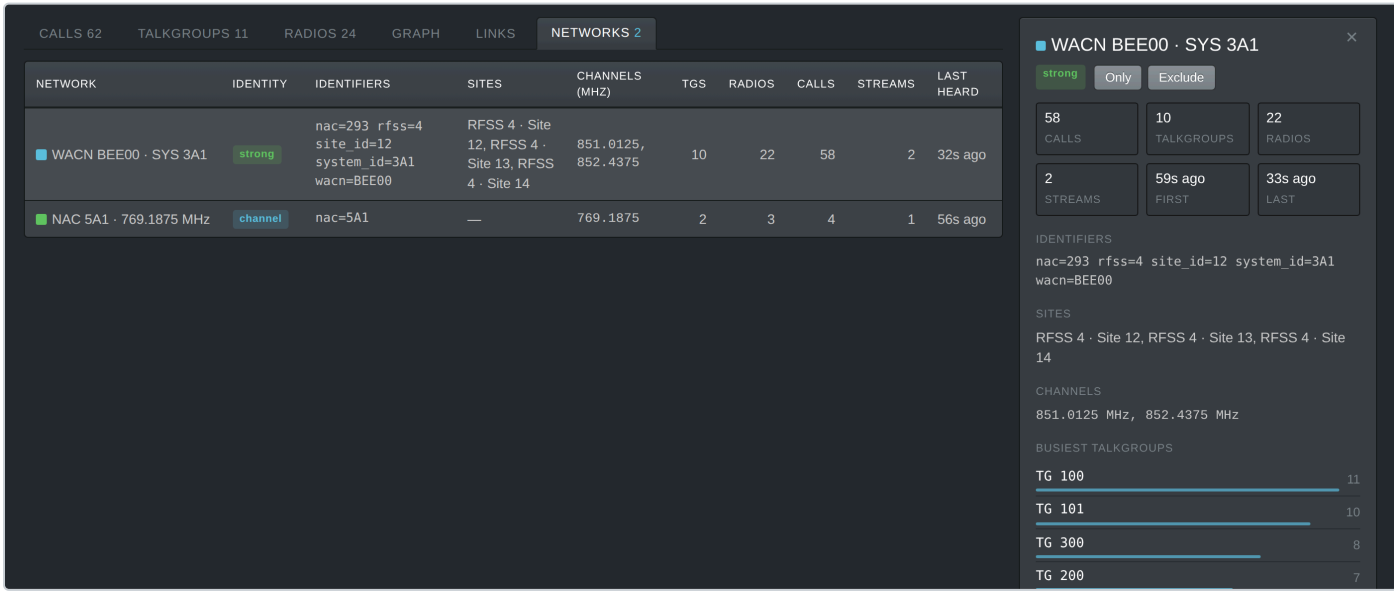



Figure 6: Networks: a strongly identified P25 system (three sites, two channels) and a NAC heard on a known channel; the system's details on the right.

## Merging networks

A network without a system id is named by its code and channel (`Color Code 1 · 436.6250 MHz`). If the frequency reported for the same channel drifts — receivers or sessions with different tuning offsets — the same repeater can show up twice (`… 436.6250 MHz` and `… 436.6275 MHz`), with its talkgroups and radios split between the two. Merge them to see them as one:

- In a network's details, **Merge into…** lists the other networks of the protocol, likeliest first: the same code (`same code`), the nearest channel (`2.5 kHz away`) and how many talkgroups and radios they share (`3 shared`). Pick one and press **Merge**: this network becomes part of that one.
- Or pick two or more networks in the open **Filters**' Networks row and press **Merge picked (N)**: they are merged into the busiest of them.
- Or take a suggestion from **Links → Probably the same network** (below).

### Merge suggestions

The **Links** view lists pairs of networks that are probably one, with the evidence and a rating — **very likely**, **likely** or **possible**:

| Evidence | Meaning |
|---|---|
| **N calls heard on both** | The same call (same source, target and kind, within 4 s) was heard on both networks by different streams — one transmission, two frequency readings. The strongest sign: *very likely*. |
| **same color code (1)** (or NAC, RAN, …) | Both carry the same short code. |
| **2.5 kHz apart** | Their channels are this close — less than a channel apart (real channels are 6.25 or 12.5 kHz apart). |
| **one took over when the other stopped** | They were never heard in the same period: what a retune or a changed offset looks like. |
| **talkgroups / radios shared** | Supporting evidence only: on amateur DMR, radio ids are worldwide and TG 9 is local on every repeater. |

The rating: **very likely** needs a call heard on both. **Likely** means a call was heard on both, *or* the codes match and the channels are within 2.5 kHz — two distinct systems can't share a code and sit that close, so it's one channel heard at a small offset. A matching code with a wider gap, or frequency proximity alone, stays **possible** (codes aren't unique) until a call is heard on both.

A pair is never suggested when the evidence says they are different channels: both have a system id; their codes differ; **one stream heard both at the same moment**; **different calls on the same slot at the same moment** (two transmissions at once — unless the same calls were also heard on both, which is shown as *but: different calls at the same moment N*×); or their channels are more than 6.25 kHz apart (or unknown) with no call heard on both.

**Merge** merges the less busy one into the busier. **Not the same** stops suggesting that pair in this browser (**Suggest them again** brings them back). The **Merge into…** list in a network's details is ordered by the same evidence (its rating first).

**Auto-merge.** A checkbox at the top of the section — **Auto-merge likely+ within 2.5 kHz** — merges the strongest pairs for you: those rated *likely* or *very likely* **and** whose channels are within 2.5 kHz of each other. Only those; a *possible* rating, or a larger frequency gap, is always left for you to decide, and the pairs you marked **Not the same** are never touched. Each auto-merge is an ordinary merge — it shows under **Merged networks** and **Unmerge** undoes it. It is on by default (the server sets the default with `DSD_NET_AUTO_MERGE`; see

§17); the checkbox overrides that for your browser. On the live server an auto-merge applies for everyone, exactly as if you had pressed Merge; in a file view it stays in that view. Turning the checkbox off stops further auto-merges but leaves ones already made (Unmerge them if you want them back).

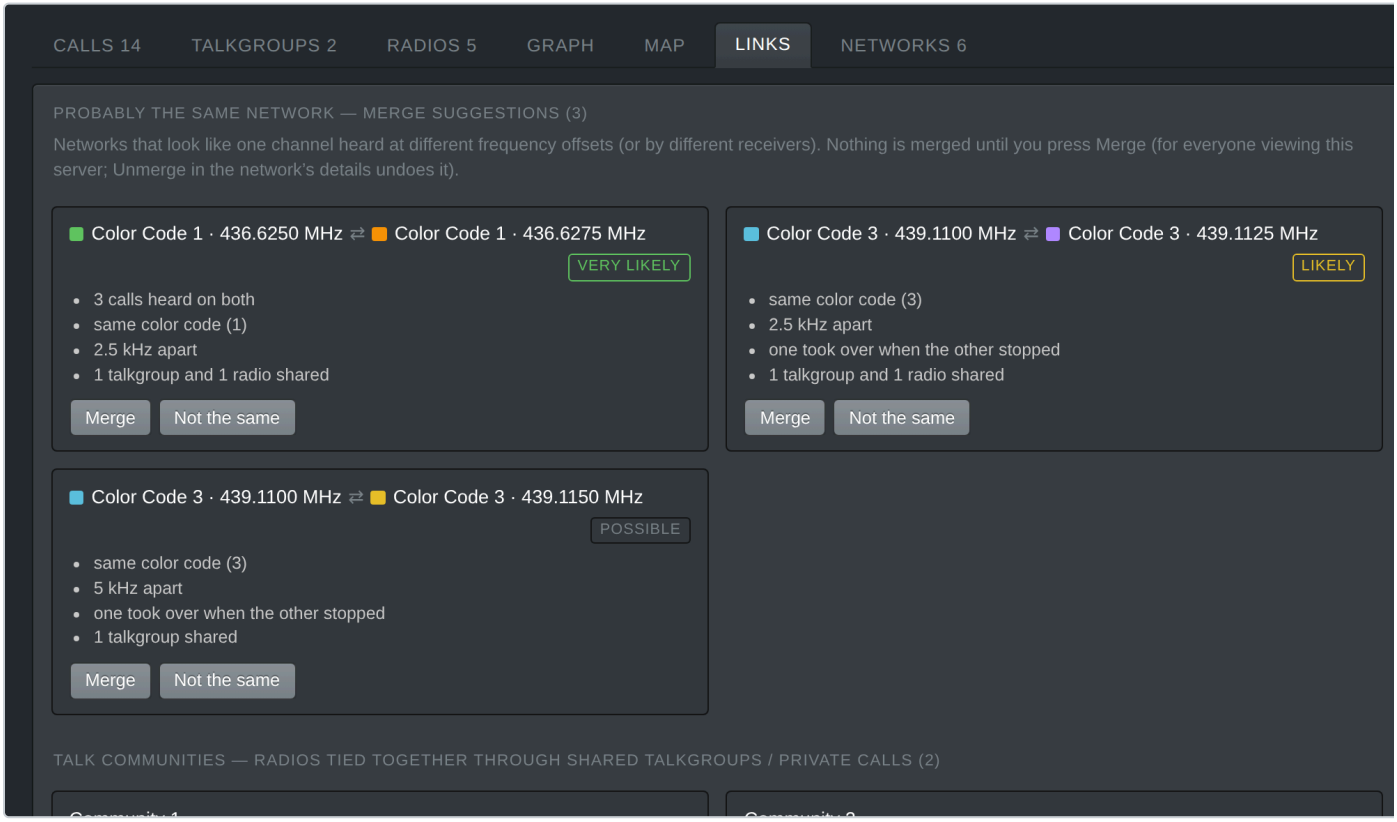



Figure 6c: Merge suggestions: a pair heard together (very likely), a hand-off 2.5 kHz apart (likely) and a pair 5 kHz apart (possible). A pair with different calls at the same moment is not suggested.

A merged network keeps the name of the one merged into, lists every channel, and adds up the counts; the Networks view marks it **+N merged**, and every view, filter, the graph and the map treat it as one network. Its details list **Merged networks** — each part with its calls and channels — with **Unmerge** for each (and **Unmerge all**). Nothing is changed underneath: the networks stay apart in the data, so unmerging gives back exactly what was there.

Merges are **shared**: everyone viewing the server sees them, they stay through **Clear** and restarts (the server keeps them in `net_merges.json` next to the recordings, or `DSD_NET_MERGES_FILE`), and exports carry them — opening or importing that export shows the networks merged too. In a file view, merging and unmerging only change that view.

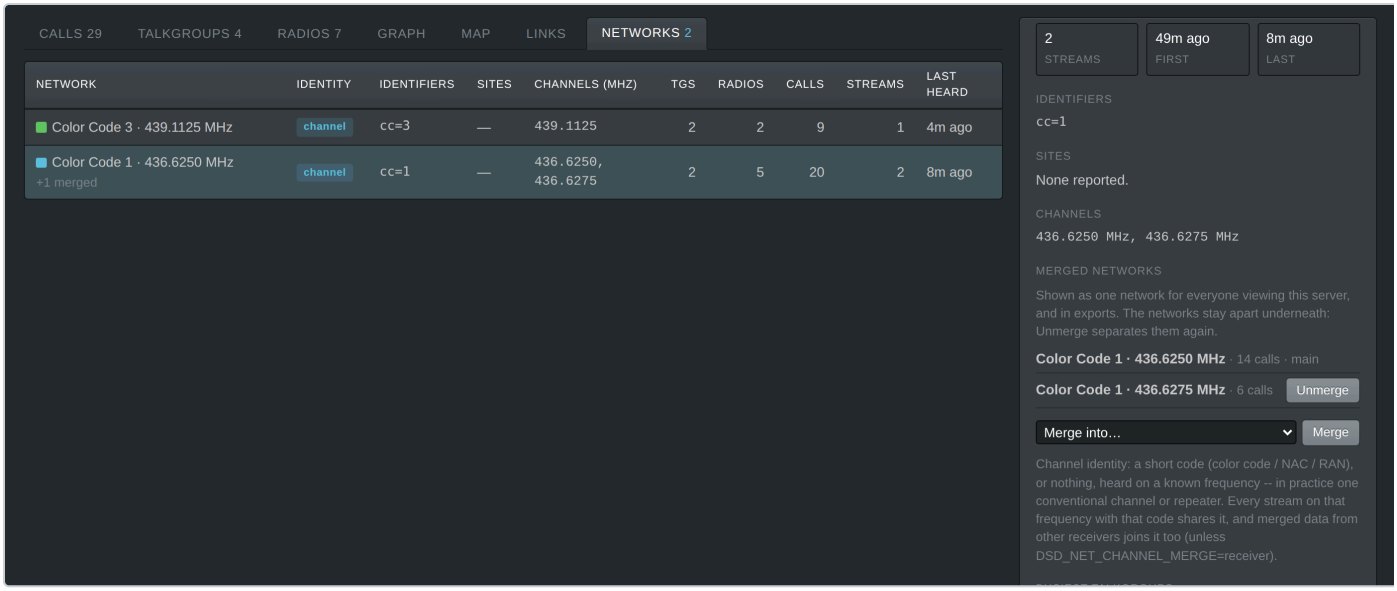



Figure 6b: Two DMR networks — one repeater heard 2.5 kHz apart — merged into one; its details list the parts with Unmerge, and Merge into… suggests the remaining network.

## 8. The details panel

Click any radio, talkgroup or network — in a table, the graph, a chip or a link — to open its details. Selecting something only shows its details and highlights it (in the graph, its neighbours): it doesn't filter the lists or the stat cards. To deselect, click the ✕ at the panel's top right, press **Esc**, click the same item again, or click an empty part of the graph. On a tablet or phone the details are a drawer: **Close** or Esc.

- **Radio**: **Only** (show only this radio; **All** undoes it) and **Exclude** (hide it; **Include** undoes it); calls, talkgroups, private peers and networks; **Talkgroups used** (by calls); **Private calls with**; **Shares talkgroups with** — other radios on its talkgroups, with how many in common; **Positions (N)** when it sent position reports — N different places (repeats of the same spot count once): the latest, linked to Google Maps; a **map** of its positions and path (oldest fix green, latest red — a sketch instead with **No map**), how far it spans and moved, **Map view**, **Download KML** (the exact track and every fix, timestamped, for Google Earth or Google My Maps' *Import*) and the fixes, newest first; **Encryption keys seen** (below); **Recent calls**, with ▶ where audio exists and, under each, its text, what a data call carried (ACK, ARS, …) and any position report.
- **Talkgroup**: **Only** (show only this talkgroup; **All** undoes it) and **Exclude** (hide it; **Include** undoes it); calls, radios, emergency / encrypted counts, networks; **Encryption keys seen** (below); **Radios on this talkgroup**; **Linked talkgroups (shared radios)**; **Recent calls**.
- **Network**: **Only** (show only this network; **All** undoes it) and **Exclude** (hide it; **Include** undoes it); counts; identifiers; sites; channels; **Merged networks** with **Unmerge**, and **Merge into…** (Merging networks); first and last heard; **Encryption keys seen** (below); its busiest talkgroups and radios.
- **Encryption keys seen** (radio, talkgroup, network; only when there are any): each key its encrypted calls announced -- the algorithm (named per protocol: P25 *AES-256*, *ADP (RC4)*, *DES-OFB*…; DMR *RC4 (EP)*, *AES-256*…; NXDN *DES*, *AES*, *Scrambler*) and the **key id** -- with how many calls used it. Keys belong to a talkgroup or channel rather than a radio: every radio on a talkgroup uses its key. (P25, DMR and NXDN, from dsd-fme; a call flagged encrypted without its key id decoded counts as encrypted only.)

- **Loading a key.** On a **network's** details (live view), each key id has **Add key…** — type the key (hex, up to 64 digits) and **Save**. It then shows ✓ **key loaded**, with **Replace key** / **Remove** / **Show key** (reveals the stored value, fetched on request; **Hide key** puts it back). A key is stored per network and key id; talkgroup and radio panels show ✓ **key loaded** for reference but are managed on the network. When the algorithm's key length is known (e.g. AES-256 = 64 hex digits, AES-128 = 32), a shorter value is padded with leading zeros, so `12345` becomes `0000…012345` — you need not type the leading zeros.
- **Using it.** The server applies the stored keys to **its own** decoder (the dsd-fme backend): when a stream starts, this protocol's keys are handed to dsd-fme as a `-K` key list, so an encrypted call whose key id matches a stored key is decoded in the clear — its audio plays and records like any other. dsd-fme matches a key by the **key id the call announces**, so clear calls are unaffected. Keys added while a stream is running take effect the next time that stream starts; to apply them to the streams running now, use ↻ **Apply to live streams** in the Keys tab (it restarts each running decoder, so its audio glitches briefly — the decode client stays on the same audio port). To decrypt in a dsd-fme you run yourself instead, ⬇ **Download key list** gives the network's keys as a dsd-fme hex key list (`keyid,key`) to point your own decoder at with `-K`. Keys are kept on the server across restarts (`DSD_NET_KEYS_FILE`, default `net_keys.json` in the recordings folder, written readable only by the server's user). A key value is never sent in `/net.json` and never put in an export; it leaves the server only when the operator asks for it — **Show key** in the panel, or the key-list download (and when the server hands its own decoder the list). (`DSD_NET_NO_APPLY_KEYS=1` keeps the keys for download only and does not hand them to the server's decoder.)
- Only enter keys for systems you are **authorized to monitor**.
- **Keys tab.** Each protocol tab gains a **Keys** sub-tab when it has any encryption key ids (seen or loaded) or any encrypted call. It lists them grouped by network with the same add / replace / remove and **Download key list** as the network details, so you can manage every key for a protocol in one place. A key id with a key loaded but not yet heard in a call still appears (marked **not heard yet**), so a key you added by hand is always visible with its **Replace key** / **Remove** controls. A key stored for a network **not heard this session** (e.g. kept across a restart, before any traffic on it) still shows too — under a · **not seen this session** label, fully manageable — and its protocol gets a tab even with no traffic yet, so stored keys are never hidden just because their network is quiet. **Download all ()** gives a zip of per-network key lists (one `-K` CSV per network, so there are no key-id collisions), and **Remove all** clears every loaded key for the protocol (with a confirm). A value is shown only when you ask for it with **Show key**.
- **Adding a key by hand. + Add a key manually** (Keys tab) opens a short form — pick the **network**, type the **key id** (hex, e.g. `1` or `666A`), optionally choose the **algorithm**, and enter the **key value** (hex, up to 64 digits) — so you can load a key you already have before an encrypted call for that key id is ever heard. The key is stored exactly like one added from a key id in a call, and the download key lists and your decoder pick it up the same way.
- **Basic Privacy (DMR).** DMR Basic Privacy is different from the keys above: its "key" is a **number, 1–255**, that selects a well-known key, and it carries **no key id**. Set it through the same **+ Add a key manually** form: choose **DMR Basic Privacy** as the algorithm and the key-id box greys out while the value box becomes the **1–255 number**. It is stored per network and applied to the server's own decoder with dsd-fme's `-b`, matched to the network's **frequency** when a stream starts; the networks that have one appear in a **Basic Privacy** list on the DMR Keys tab, each with **Clear**. Because BP has no key id, a decoder applies **one** BP key per channel; dsd-fme applies it only to a call flagged encrypted that has no key id, so clear and key-id (EP/AES) calls are unaffected. Set it on channels you know use BP. BP works alongside the key list and Enhanced Privacy keys on the same channel with the dsd-fme this server ships (it carries

a small fix for that; a dsd-fme you build yourself needs the same patch — see the README). The number isn't secret (the keys themselves are well known). As with the key list, a change takes effect when a stream next starts, or now with **↻ Apply to live streams**.

- **Motorola Enhanced Privacy (DMR), any key id.** An EP (ARC4) key normally goes in with its key id like any other key (algorithm **RC4 (EP)**). When you have a network's EP key but not its key id, choose **Motorola Enhanced Privacy (any key id)** in the same form: the key-id box greys out and the value is the **40-bit key, up to 10 hex digits**. It is stored per network and, matched to the network's **frequency** when a stream starts, handed to the server's own decoder as the key for **every** key id (00–FF) in its key list; a key you stored for a specific key id still wins for that id. Clear calls are unaffected, but an encrypted call on that channel whose key id has no key of its own is tried with the EP key, so set it only on channels you know use EP. Unlike a BP number it **is** secret: it is kept with the keyring and never shown again — the networks that have one appear in a **Motorola Enhanced Privacy** list on the DMR Keys tab, each with **Clear**. It is not included in a network's key-list download. A change takes effect when a stream next starts, or now with **↻ Apply to live streams**.
- **TYT Enhanced Privacy (DMR).** TYT radios (MD-380 / MD-UV380 and kin) and the Baofeng DM-1701 call their AES-128 privacy "EP" too, but it is a different scheme: a **32-hex-digit key** such as `00000000000000000000000000012345`, with **no key id and no encryption flag** in the call. Choose **TYT Enhanced Privacy (AES, 32 hex)** in the same form; the key-id box greys out and the value takes the key (shorter keys are zero-padded, spaces are ignored). It is stored per network and, matched to the network's **frequency** when a stream starts, applied to the server's own decoder with dsd-fme's `-5`. Because nothing in the call says it is encrypted, the decoder applies it to **every** voice frame on that channel: a clear call there is garbled, so set it only on channels where all traffic uses this key. It works alongside BP, Motorola EP and the key list. Like Motorola EP it is secret — kept with the keyring, never shown again, listed in a **TYT Enhanced Privacy** list with **Clear** — and it takes effect when a stream next starts, or now with **↻ Apply to live streams**.

- **TYT / Anytone Basic Privacy and TYT / Baofeng / Retevis Advanced Privacy (DMR).** More keys for radios that send no key id or encryption flag, set the same way from the form's algorithm list:

| Option | Key | Radios | dsd-fme |
|---|---|---|---|
| **TYT Basic Privacy** | 16 bits, up to 4 hex digits | TYT MD-380 / MD-UV380 | `-2` |
| **Anytone Basic Privacy** | 16 bits, up to 4 hex digits | Anytone AT-D878 / D578 | `-A` |
| **TYT Advanced Privacy** (PC4) | 128 or 256 bits: up to 32, or exactly 64, hex digits | TYT MD-UV380 / 390 | `-!` |
| **Baofeng Advanced Privacy** (PC5) | 128 or 256 bits, as above | Baofeng DM-1701, DM-32 | `-+` |
| **Retevis Advanced Privacy** (RC2) | 128 or 256 bits, as above | Retevis RT3S / RT90 | `-@` |

Like TYT EP, each is stored per network, applied to **every** voice frame on the network's frequency (a clear call there is garbled), kept secret, and listed under its own heading on the DMR Keys tab with **Clear**. A channel can have only **one** of these "all voice" keys (TYT EP, TYT or Anytone BP, or an Advanced Privacy key) — two can't both be right — so setting one replaces any other on that network. Basic Privacy and Motorola EP are separate and stay as they are. TYT BP, like DMR Basic Privacy, needs the small dsd-fme fix the server's dsd-fme carries to keep working after the first call when other keys are stored (see the README for a dsd-fme you build yourself).

## 9. Searching, filtering and sorting

- The **search box** matches radio ids, talkgroup ids, aliases, SMS text, frequencies (e.g. `460.17`) and encryption (a key id such as `0x666A`, or an algorithm such as `AES`), in every view, including the graph and links.
- The **Filters** section gathers every filter. Each limits every view, the graph and the stat cards, and they combine:
- **Networks** — click network chips to show only those (one or several; each click adds or drops one). In a network's details, **Only** picks just that one and **Exclude** hides it (its calls, talkgroups and radios). The Networks view still lists every network, so an excluded one can be included again. With two or more picked, **Merge picked** shows them as one network (Merging networks).
- **Talkgroups** — from a talkgroup's details (**Only** / **Exclude**), or by typing ids into the box in the open Filters' Talkgroups row: **Pick** (or Enter) adds them to the picked ones, **Exclude** excludes them. Several at once work (`16 12`, `TG 16, TG 12`), and the box suggests the talkgroups heard. Picking talkgroups shows their calls and the radios that used them (private calls, which have no talkgroup, are hidden); excluding one hides its calls, and radios heard only on it.
- **Radios** — the same, from a radio's details or the Radios row's box, which also takes an alias. Picking a radio shows its calls (both ends of a private call count) and the talkgroups it used; excluding one hides it and its calls.
- **Community** — set by clicking a community on the Links view; it shows only that community's radios and talkgroups (and their calls). It has no box of its own — the chip's ✕ (or clicking the community again) drops it.
- **Search** — the search box's text, shown here so it can be cleared.

An id not heard yet is still added (the page says so), so you can filter for a talkgroup or radio before it turns up. A picked item's chip is highlighted and an excluded one's is dashed and struck through; click a chip to take it out of the filter, the **All …** chip of a row to clear that row, or **Clear all** for everything. The filters reset when you switch protocol tabs, **Clear**, or open a file. - **Click a column header** to sort by it; click again to reverse. Calls can be sorted by start, duration, frequency, slot, source, target and **Audio** (calls with audio first). On a phone a **Sort** menu replaces the headers. - Filters and sorting also decide what **⤓ Audio (.zip)** saves.

## 10. Call audio

The explorer can keep each call's decoded voice so you can play it back. **Off by default.** Turn it on with ♫ **Audio** in the header, or start the server with `DSD_NET_AUDIO=1`. The server remembers the switch: after a restart, audio recording is on or off as you last left it (unless the server is started with `DSD_NET_AUDIO` set, which decides at every start).

- ▶ plays a call (press again, or ■, to stop). The call playing stays in a **bar at the bottom of the window** — time, channel, who → whom, ↓ download and the transcript — however fast the list moves. ✕ closes it. A call **still in progress** plays as it is being recorded: at the end of what has arrived, the player fetches the rest and carries on (with a brief pause), until the call is over; with speech-to-text on, it is transcribed once the call ends.
- A **seek slider** across the top of the bar shows how far in you are; click or drag it to move around (available once the call is over and its length is known — a call still in progress isn't seekable yet).
- A **spectrogram** of the whole call fills the bottom of the bar — time left-to-right, frequency bottom-to-top (0–4 kHz), loudness as colour (dark = quiet, bright = loud) — with a playhead that tracks playback; click or drag on it to seek too. It is computed **in your browser** (the audio is fetched, decoded and Fourier-transformed there), so the server does no analysis work.
- The **Spectrogram** tick in the Calls toolbar turns the slider and spectrogram on or off (on by default; your choice is remembered in this browser). With them off the bar shows just a plain progress line. The server-wide default is set with `DSD_NET_AUDIO_VIZ` (§17).
- ↓ next to ▶ downloads the call's WAV (8 kHz mono), named after the call: `call_20261002T172931Z_434.4250MHz_s1_TG1_from_123.wav`.
- ↓ **Audio (.zip)** saves the audio of the calls **listed, in the order shown** (so filter, search or sort first), up to 25 MB, with a `calls.csv` of time, duration, network, MHz, slot, from (and alias), to, flags and any transcript (Appendix D lists every column). Handy for sending examples.

What gets recorded:

- One file per call. DMR's two timeslots are recorded separately; audio heard up to 1 s before the call was decoded is included; a call heard by two receivers is recorded once.
- **Never**: silence (no file without audible audio; recordings under 0.2 s are deleted), data calls (SMS, private data), and **encrypted calls** (unless the session has the key). An encrypted call heard without a key can instead be *kept for later* decryption — see Keeping encrypted calls for later.
- Disk use is capped (1 GB by default, about 18 hours of speech); the oldest files go first. A call keeps its place in the list for as long as its audio is on disk — so this disk limit, not the 10000 call count, is what bounds how much recorded audio stays reachable. When a call's audio is pruned, its play button goes away and the call then ages out by the ordinary count.
- Audio stays on the server that recorded it. A plain export notes which calls had audio; **Export ▾ → Explorer data with audio (.zip)** also packs the audio, and opening that zip plays it (§12). **Importing** that zip into the live view uploads its audio to this server too, so the imported calls play alongside your own (see below).

Rules on keeping intercepted communications differ from place to place; check what applies to you before switching audio on.

### Keeping encrypted calls for later (Pending)

An encrypted call heard on a stream that has **no key** can't be decoded while it happens, so its voice is normally discarded. If you expect to obtain the key afterwards — say an operator loads it on the radio but hasn't told you yet — the **Pending ▾** menu keeps the call so it can be **decrypted after the fact**.

- **What is kept.** Not "the ciphertext" — dsd-server doesn't decrypt anything itself; the external decoder (dsd-fme) does, from the discriminator audio the server feeds it, using the key it was given at start. The key only changes how that same audio is decoded, so the server keeps **the discriminator audio of the session** (48 kHz mono, about 96 KB/s). Replaying it through the decoder with the key present then recovers the voice — no re-tuning needed.
- **Turning it on. Pending ▾ → Capture encrypted calls without a key.** Capture begins only once an encrypted call with no key is actually heard on a stream, and runs for the rest of that stream's session; a stream that already has a key decodes live and is never captured. The switch is remembered across restarts, and can be set from startup with `DSD_NET_PENDING_CAPTURE=1` (`=0` never). You must enable it **before** the calls happen — it can't go back in time.
- **The list.** The menu shows each kept capture: when it started, the protocol and channel, its length and size, and the **alg:key-id** pairs seen in it (the ids a matching key must cover). Captures are kept until you remove one (✕) or **Clear all captures** — they are not pruned by age. A generous disk cap (`DSD_NET_PENDING_MAX_MB`, default 8192 MB; `0` = no cap) evicts the oldest only if reached. Captures live under `DSD_NET_PENDING_DIR` (default `net_pending/` in the recordings folder) and never contain key values.
- **Decrypting a kept call.** Load the matching key (Keys tab, or **+ Add a key manually**). When a key you add can decrypt one or more stored captures, a message says so — open **Pending ▾** and click **Decrypt** on a capture. The server replays that capture's audio through a fresh decoder with the keyring now loaded; the recovered calls appear in the explorer like any other (with their audio, and dated to the capture's original time, not now). While a decrypt runs the button shows **decrypting…**; it does one capture at a time, and a message reports how many calls were recovered when it finishes. Decrypting turns on call-audio recording (§10) so the recovered voice is kept. A capture whose key you don't have yet still decrypts nothing — load the key first.

Only capture and decrypt traffic on systems you are **authorized to monitor**, and mind the same local rules that apply to recording audio.

## 11. Speech-to-text

With **Transcribe on play** ticked, playing a call also turns its speech into text. The text shows in the bottom bar, then under the call in the list, and goes into the audio zip's `calls.csv`.

- It runs **in your browser** (OpenAI's Whisper via Transformers.js); the server only provides the files. Someone has to fetch them onto the server once with `tools/get_asr_assets.sh` (see the README). Without them the bar says so and offers to load them from the internet instead — only if you choose to.
- **Language**: pick the spoken language (default English). "Detect language" guesses, poorly on short calls.
- **Model**: shown when the server has more than one. `small` (the default) is far more accurate on radio voice than `base` or `tiny`, but slower.

- **Speed**: the first call of a visit also loads the model (about 250 MB for `small`; cached afterwards). Then roughly 5–15 s per call on a desktop. Pages opened over plain `http://<server-ip>` run on one CPU thread and take about twice as long as HTTPS or `localhost`.
- **Accuracy**: good on clear speech, unreliable on short or garbled calls. Whisper sometimes invents text on silence or noise ("you", "Thanks for watching") or gets stuck repeating a phrase; the explorer drops those and shows "(no clear speech recognized)". Treat transcripts as a hint.
- Transcripts are kept in **your browser** (the last ~1000), not on the server — another browser starts without them. Replaying a transcribed call is instant.

## 12. Saving, reopening and combining data

- **Export ▾ → Explorer data (.json)** saves the networks, talkgroups, radios, every association with its counts, and the recent calls. **Open…** it later (or drop it on the page) to browse it offline, with every view working; times are shown relative to the export. A blue bar says you are in a file view; **Back to live** returns.
- **Export ▾ → Explorer data with audio (.zip)** is the same export plus every call's audio (`audio/*.wav`), in one zip. **Open…** it (or drop it on the page) to browse it with the audio playable — ▶, ⭳ and speech-to-text work as live; opened together with other exports, its calls keep their audio in the merged view. The zip is built in the browser and stops adding audio at 1 GB (it says how many were left out). **Import…** takes the zip too: the export joins the live view and the page then uploads each call's audio to this server (*Uploading audio 12 / 40…*), so imported calls get ▶, ⭳ and speech-to-text like your own. The report line says how many calls got audio; hover the import in the imports bar for *audio for N of M recordings*. Only audio a call in that import names is kept (anything else in the zip is ignored), it is deleted when the import is removed (✕, **Remove all**, **Clear**, or a newer export of the same run replacing it), and imports — and so their audio — don't survive a server restart. Imported audio has its own cap (`DSD_NET_IMPORT_AUDIO_MAX_MB`, 1 GB by default); past it, the rest is left out and the report says so. A plain `.json` export has no audio to upload.
- **Only what the filters show** — a tick at the top of the **Export ▾** menu, offered only while a filter is in force (a network, talkgroup or radio filter, a search, or a pinned community). With it on, the two data exports above — **Explorer data (.json)** and **with audio (.zip)** — save just the **current protocol**, and within it only the networks, talkgroups, radios and calls that pass the filters (the same ones the lists show). The display-only toggles (**With audio only**, **Hide signaling**) don't narrow it — they only shorten the on-screen list. The file name gets `_filtered`, it carries a `filtered` marker, and it re-opens and imports like any other export. Off (the default), the exports hold everything on the server, every protocol. The graph and positions exports below aren't affected (Positions already follows the filters).
- **Export ▾ → Association graph (.graphml)** is for graph tools (Gephi, Cytoscape, yEd, networkx).
- **Export ▾ → Positions (.kml)** saves the position reports of the radios listed (this protocol, with the filters): a track and timestamped points per radio, for Google Earth or Google My Maps (*Import*). Each radio keeps its latest 100 distinct positions; they are in the explorer data too.
- **Open…** with **several files** shows them merged, read-only, with a report of what happened to each file and a **Save merged** link.
- **Import…** adds exports to the **live** view — e.g. other receivers' exports — shown together with the live data. A bar lists the imports, each removable (✕, or **Remove all**). The ✕ at the bar's right end only hides the bar: the imports stay included, and a small **N imports** beside the protocol tabs brings it back. This

browser remembers it until the imports change (a new import shows the bar again). New traffic keeps arriving on top; **Export** then saves the combined view.

When files are combined, talkgroups and radios join by id within a protocol; networks join by their system id, or by code and channel. A call two receivers both heard counts once (**2 RX**). The same data is never counted twice: duplicate or overlapping exports are skipped or refused, with the reason shown. Name each receiver with `DSD_SERVER_NAME` so its networks are easy to tell apart. `net-merge` does the same from the command line. Exports carry their network merges; combined files keep them all (a merge made on this server wins over an imported one).

## Comparing to a baseline (what's new)

**Compare…** loads a saved export as a *baseline* — yesterday's run, say — and flags what is **new** in what you are looking at now (the live view, or another export you have open). It does not change the view: a green bar under the stat cards sums it up — **+N radios · +N talkgroups · +N networks** new since the baseline, any **newly encrypted** talkgroups (encrypted now, clear or absent in the baseline), and how many radios/talkgroups in the baseline were **not heard** this time. Click a count to jump to that list; the new talkgroups, radios and networks there (and in their details) carry a green **NEW** badge.

With **New first** ticked in the bar (on by default while comparing), the Calls, Talkgroups, Radios and Networks lists float their new rows to the top — the column sort you pick still orders within the new and the older groups. In the Calls list, a call counts as new when a new radio or talkgroup is at either end, and the new one wears the badge. Untick it to sort normally.

A **Changes** tab appears while comparing (the page opens on it when you load a baseline) with the full day-over-day diff for the current protocol, grouped into sections — each collapsible, each row a link to the entity, longest first:

- **New talkgroups / New radios** — not in the baseline, with their call count.
- **Newly encrypted** — talkgroups or radios encrypted now that were clear or absent before.
- **New talkgroup use** — a radio already in the baseline now talks on a talkgroup it didn't before (both already known): a new association, not just a new entity.
- **New private-call pairs** — radios already known that started private calls with each other.
- **Busier since baseline** — entities heard in both runs with more calls now (shown as *before → after*).
- **New networks** — not in the baseline (subject to the network-identity caveat below).
- **Not heard (in baseline only)** — talkgroups and radios that were in the baseline but aren't in the current view.

**Save report (.json)** on that tab writes the whole comparison (ids and counts, no key values) to a file.

"New" means not present in the baseline for that protocol. Talkgroups and radios match by id, so those — and the associations built from them — are exact; a network matches on its strong identifiers (system id) and falls back to its channel key, which **can drift between days** on weakly identified systems, so a "new network" is worth a glance rather than trusting blindly (radios and talkgroups are the reliable signal). The baseline is held in your browser only (it survives the page's own reloads) and compares against whatever protocol tab you are on; it holds the baseline's networks, talkgroups and radios, not its calls. Load a different export to replace it, or

**Compare… → Comparing ✕** (or the bar's **Clear**) to stop. The Changes tab reports over all of the current protocol's data (the Filters don't narrow it). A scheduled, automated day-over-day watch with alerts is a possible next step.

## 13. Recording a session for troubleshooting

If something in the explorer looks wrong (phantom calls, calls cut short, networks split or merged wrongly), a **recording** lets it be reproduced exactly:

**Record** is a developer tool, so it is hidden from regular users. To show it in your browser, open the explorer once as `http://<server>/net?dev=1` — the browser remembers it (`?dev=0` hides it again). A server started with `DSD_NET_DEV=1` shows it to everyone.

1. Click **Record** (accept the offer to clear first — the replay is then exact).
2. Let the problem happen, then click **Stop recording** and **Download** (`net_<UTC>.jsonl.gz`).
3. Send the file. `net-replay` rebuilds precisely what the explorer showed, and can show the state at any moment or produce an export from it.

A recording holds the decoder's output (not audio or IQ), compressed — around half a megabyte per minute on a busy multi-channel server.

## 14. Tablets and phones

- Touch targets are finger-sized and the page scrolls as one.
- Below about 1050 px wide the details open in a **drawer** from the right (a **bottom sheet** on a phone) with a **Close** button.
- On a phone: the header's actions move into **☰ Menu**, calls are listed as **cards**, other tables become cards with a **Sort** menu, and the stat cards shrink to a strip.
- The graph has pinch-zoom and − / + buttons.
- Explanations that a mouse shows on hover (badges, identity levels, Calls / s) appear when tapped.

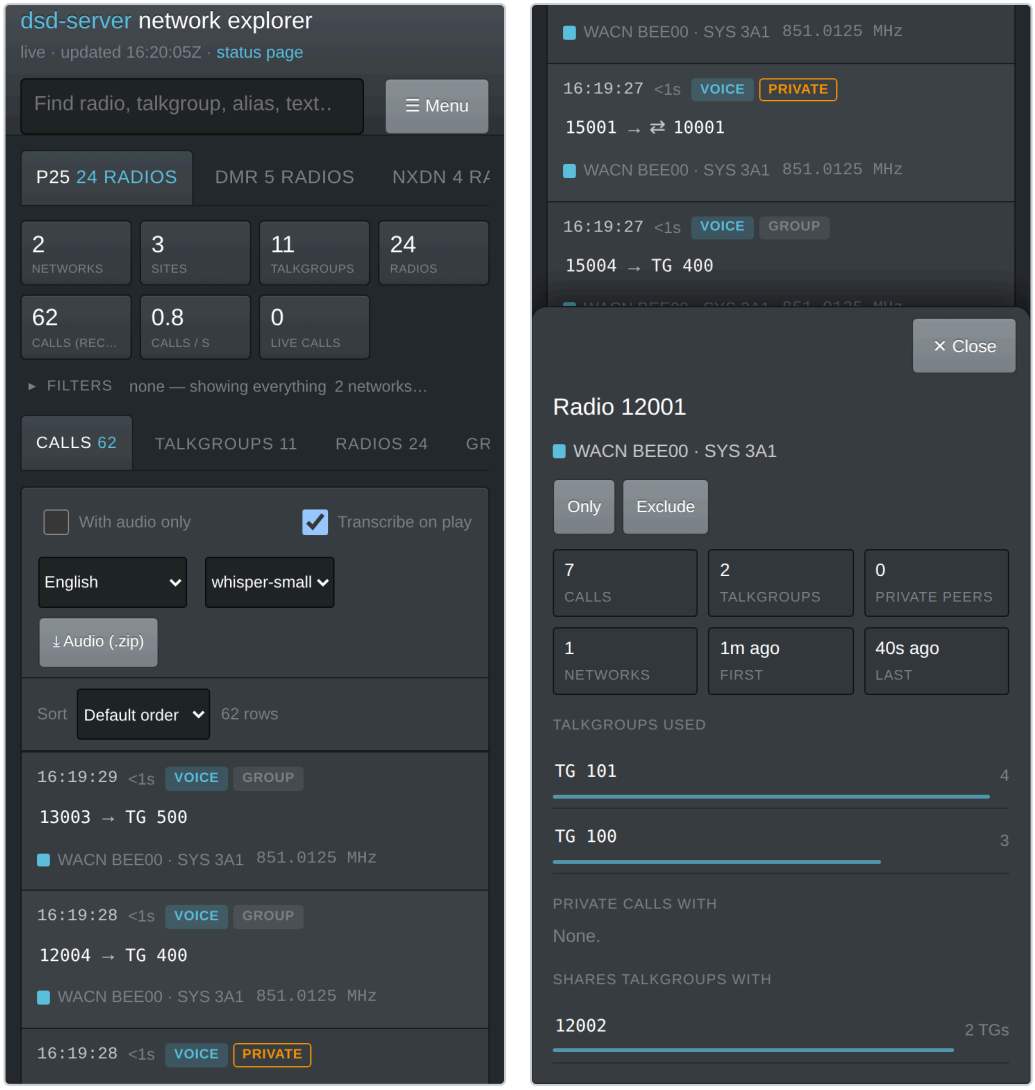



Figure 7: On a phone: calls as cards (left) and a radio's details in a bottom sheet (right).

## 15. Limits: what is kept, and for how long

- Everything is in the server's memory: it resets when the server restarts or someone presses **Clear**. Export (or record) what you want to keep.
- **Calls**: the newest 10000 **without audio** per protocol (`DSD_NET_MAX_CALLS`), plus every call whose recorded audio is still on the server (bounded by the audio disk limit, not this count — §10). Talkgroup / radio / network counts and associations are kept for everything heard, not just the listed calls.
- **Radios / talkgroups / networks**: up to 3000 / 1500 / 200 per protocol; the least recently heard go first.
- When a session ends, a network that never carried a call (and radios known only through it) is dropped if it never identified itself — `Unidentified`, or a code tied to that one stream (`Color Code 4 · stream 3`) — or if a fuller identity covers it (a P25 NAC, or a SYS without its WACN, once the system's network is known), unless another running session is still on it. A network that identified itself — a system id, or a code on a known channel — stays, calls or not: a quiet control channel doesn't disappear when its client reconnects. When a session retunes, the old channel's call-less networks go.

## 16. Troubleshooting

| Symptom | Likely cause / what to do |
|---|---|
| A session runs but nothing appears | The explorer only shows streams that decode real traffic. Check the status page: is the protocol right, is anything being decoded? |
| `Unidentified · …` network | No identity decoded yet (often at the start of a session, or on a quiet channel). It is not listed until a call is heard on it or an identity is decoded, then it appears (and updates as more is heard). |
| One system appears as several `Color Code n · stream N` networks | The client didn't send the frequency, so short codes can't be trusted to join streams. Send `center_freq` in `start`. Meanwhile **Links** → **Seen on more than one network** shows the shared radios and talkgroups. |
| Many very short calls with no source | Usually decoder lines the explorer misread as calls. Make a **recording** (section 13) and send it — that is how the Capacity Plus channel-status issue was found and fixed. |
| The list moves too fast to read | The header's **Pause** freezes the page; or narrow the list with the search box or the **Filters**. |
| ▶ buttons don't appear | Audio recording is off — press ♫ **Audio**. Encrypted and data calls never have audio. |
| "Speech-to-text needs its files on the server" | Run `tools/get_asr_assets.sh` on the server (or point `DSD_NET_ASR_DIR` at a copy), then reload the page. |
| Transcripts are slow | Open the page over HTTPS or as `localhost`, use a faster PC, or pick `base` in the model list (faster but much less accurate). |
| Transcript is wrong | Try the right language, or `small` / `small.en` if a smaller model is selected. Short or poor-signal calls are often beyond any model. |

## 17. Settings reference

Server environment variables that affect the explorer (details in the README):

| Variable | Default | Effect |
|---|---|---|
| `DSD_SERVER_NAME` | host name | This receiver's name in exports and merged views. |
| `DSD_NET_MAX_CALLS` | 10000 | Audio-less calls kept per protocol. Calls with recorded audio are kept beyond this for as long as their audio is on disk (bounded by `DSD_NET_AUDIO_MAX_MB` / `_MAX_AGE_H`, not this count). |
| `DSD_NET_FREQ_STEP_HZ` | 1250 | Channel raster frequencies are snapped to. |
| `DSD_NET_MERGES_FILE` | `net_merges.json` in the log dir | Where the network merges are kept across restarts. |
| `DSD_NET_AUTO_MERGE` | on | `0` (or `off`) sets the default off for the **Auto-merge likely+ within 2.5 kHz** tick (merge suggestions); a browser can still override it. The judging and merging run in the page. |

| Variable | Default | Effect |
|---|---|---|
| `DSD_NET_CHANNEL_MERGE` | (merge) | `receiver` keeps different receivers' channel networks apart when merging. |
| `DSD_NET_AUDIO` | (as last left) | `1` records call audio from startup, `0` never at startup; unset, the ♫ **Audio** switch is remembered across restarts. |
| `DSD_NET_SETTINGS_FILE` | `net_settings.json` in the log dir | Where remembered switches (♫ Audio) are kept. |
| `DSD_NET_AUDIO_DIR` | `net_audio/` in the log dir | Where call audio goes. |
| `DSD_NET_AUDIO_VIZ` | on | `0` (or `off`) hides the player bar's seek slider and spectrogram for everyone, leaving a plain progress bar; a browser can still turn them back on with the **Spectrogram** tick. All drawing is client-side regardless. |
| `DSD_NET_AUDIO_MAX_MB` / `DSD_NET_AUDIO_MAX_AGE_H` | 1024 / none | Audio disk cap and maximum age. |
| `DSD_NET_PENDING_CAPTURE` | (as last left) | `1` keeps encrypted calls heard without a key (for later decryption) from startup, `0` never; unset, the **Pending** switch is remembered across restarts. |
| `DSD_NET_PENDING_DIR` | `net_pending/` in the log dir | Where kept encrypted-call audio goes (no key values). |
| `DSD_NET_PENDING_MAX_MB` | 8192 | Disk cap for kept captures (`0` = no cap); the oldest go first when reached. Captures are otherwise kept until cleared by hand. |
| `DSD_NET_IMPORT_AUDIO_MAX_MB` | 1024 | Cap on audio uploaded with imported "export with audio" zips (kept in `imported/` under the audio dir, emptied at startup). |
| `DSD_NET_KEYS_FILE` | `net_keys.json` in the log dir | The encryption keyring (key values; owner-only). |
| `DSD_NET_NO_APPLY_KEYS` | (unset) | Set to keep the keyring for download only — do not hand it to the server's own dsd-fme decoder. |
| `DSD_NET_ASR_DIR` | `net_asr/` in the log dir | Speech-to-text files (`tools/get_asr_assets.sh`). |
| `DSD_NET_MAP_TILES` / `DSD_NET_MAP_ATTRIB` | (public tiles) | A map tile server of your own — a URL template with `{z}` `{x}` `{y}` (e.g. `http://maps.local/tile/{z}/{x}/{y}.png`) — and its credit line, for the Map view. |
| `DSD_NET_ASR_MODEL` / `DSD_NET_ASR_LANG` | most accurate present / english | Default speech model and language. |
| `DSD_NET_LOG` | off | `1` records from startup. |
| `DSD_NET_DEV` | off | `1` shows the developer tools (**Record**) to everyone; otherwise a browser opts in with `/net?dev=1`. |

| Variable | Default | Effect |
|---|---|---|
| `DSD_NET_LOG_DIR` / `DSD_NET_LOG_MAX_MB` | IQ-capture dir, else working dir / 1024 | Where recordings go, and their size cap. |

---

# Appendix A: How associations are built

This is a high-level description of what the explorer does with the decoder's output. It works the same way for every protocol; Appendix B covers the differences.

## A.1 Inputs

Each decode session is a **stream**: one receiver channel decoding one protocol. The decoder (dsd-fme or the TETRA decoder) prints lines for sync bursts, call headers, voice frames, talker aliases, SMS, and system broadcasts. dsd-server parses each line into an event — its kind (sync, call, voice, data, message …), slot, source id, target id, alias, text, and identity tokens such as a color code or system id. The explorer is fed every event of every stream, including lines clients never see.

Everything is kept **per protocol family**. Ids are only ever compared within a family.

## A.2 Which network a stream is on

Identity and calls arrive on **different lines**: a call header carries only source and target, while the system's identity comes from sync bursts or system broadcasts. So each stream remembers the identity it has decoded so far, and each call is credited to its stream's current network. Identity comes in three strengths:

- **Strong** — a real system id that is unique in practice: P25 WACN + System ID, DMR network id (Tier III / Capacity Max), NXDN system code, TETRA MCC + MNC, EDACS system id. Every stream that hears the same strong id is the same network, whichever receiver or session it came from.
- **Weak** — a short code that unrelated systems routinely share: DMR color code (16 values), P25 NAC, NXDN RAN, dPMR channel code, D-STAR repeater callsign, YSF downlink id. A weak id alone is **not** treated as a network identity: each stream gets its own network (`Color Code 1 · stream 3`), and shared radios and talkgroups appear under **Links** as evidence instead of being merged silently.
- **Channel** — a weak id (or none) **plus the channel frequency** sent by the client. A short code on one channel is in practice one repeater or conventional channel, so every stream on that frequency with that code shares one network (`Color Code 1 · 434.4250 MHz`). A stream on a known channel with nothing decoded yet is `Unidentified · 434.4250 MHz`. Such a bare `Unidentified · …` network (a receiver is tuned there but nothing has been decoded) is **not listed** -- not in the network list, the counts, filters or merge suggestions -- until a call is heard on it or an identity is decoded; its protocol tab still shows while a stream is receiving on it.

Refinements:

- **Upgrades**: a stream's identity can grow (unidentified → NAC → SYS → WACN/SYS). Its earlier, less specific network is then folded into the new one when only that stream fed it, or when the new identity strictly contains the old.
- **Retunes**: a *different* value for an identifying id (another color code, another system id) means the stream is now on another network; it starts its identity over.

- **Placeholders**: weak codes must be seen **twice in a row** before they are believed (dsd-fme prints placeholders such as `Color Code=00` at the start of a burst). Strong ids, from decoded control messages, are taken at once.
- **Neighbour sites**: adjacent / neighbour-site broadcasts add a site to the network but never change the stream's own identity or site.
- **Sites**: site ids (P25 RFSS/Site, DMR Site, NXDN site code / location, TETRA location area) are recorded as the network's sites, not as separate networks.

### A.3 From decoder lines to calls

A call is assembled from many events:

1. **Which call an event belongs to.** Calls are tracked per stream, per **slot** and per **target**. An event with a target joins the open call on that slot and target; one without a target joins the slot's latest call. This keeps two DMR timeslots apart, and keeps several grants interleaved on a control channel apart.
2. **New call.** A call starts at a call header, voice or data event with a source or target. A sync burst alone never starts one. A new call on a TDMA slot ends the previous call on that slot. A **new talker** (different source) on the same target starts a new call.
3. **End of call.** A call ends after **4 seconds** without events, or when replaced as above.
4. **What it collects.** Source and target (filled in as they are decoded — the same call is recognised with or without a slot marker, or before its target is known), talker alias, SMS text, and flags: **voice** (voice frames / voice sync), **data** (data headers, SMS, UDT), **emergency**, **encrypted** (a non-clear algorithm id or an encryption flag) -- with the algorithm and **key id** the call names, when decoded -- and **private**. The key is counted once per call on its network, talkgroup and talker.
5. **Private calls.** A call is private (radio → radio) when the decoder marks it so: a unit-target flag, or "Private", "Unit to Unit", "Individual", "I-Call" or "U2U" on its lines. A call first counted as a group call and then found to be private has its association moved from the talkgroup to the radio pair.
6. **Ignored.** Lines that failed their error check (CRC/FEC) are dropped — their ids are unreliable. System broadcasts (identity, LCW, status) never start calls, only refine the call in progress on their slot. **Rosters** — e.g. Capacity Plus channel status (`Private or Data Call(s) - LSN 01: TGT 17434; …`), which lists traffic on the site's *other* channels — never start, end or extend calls.

### A.4 Counting associations

When a call has both a source and a target it is **counted once**:

- **Group call**: radio → talkgroup. The radio's talkgroup count and the talkgroup's radio count go up; both are marked as heard on the call's network.
- **Private call**: radio ↔ radio. Both radios get the other as a **private peer**.
- The network's call count, the radio's and talkgroup's call counts and last-heard times are updated.

The derived views come from these counts:

- **Radios sharing talkgroups** (radio details): radios counted on the same talkgroups.
- **Linked talkgroups** (talkgroup details): talkgroups whose radios also use this one.
- **Talk communities** (Links): connected groups of radios and talkgroups, through group and private calls.

- **Seen on more than one network** (Links): radios or talkgroups counted on two or more networks.
- **Hub radios** (Links): radios on three or more talkgroups.

### A.5 One call heard twice (2 RX)

Two receivers often hear the same call — a P25 control channel's grant and the voice channel, two sites of one system, or two receivers on one repeater. When a call from a different stream has the **same network, source, target and group/private type** and is still running (within 4 s), it is the same call: the second is folded into the first (flags, alias and text combined, earliest start, latest end) and counted once, marked **2 RX**. Only one audio recording is kept.

### A.6 Combining exports

Merging (Open with several files, Import, `net-merge`) applies the same rules across files: talkgroups and radios join by id; networks join by strong id, or by code + channel (unless `DSD_NET_CHANNEL_MERGE=receiver`). A weak id with no channel stays per receiver (`… · stream 3 · rx-north`). Calls are folded as in A.5 across files. Each export records its server run and the time span it covers, so the same data is never counted twice.

Network merges are rules, not changes to the data: per protocol, a network key and the key it is shown under (`cc:1@436627500` → `cc:1@436625000`). The page applies them when it shows the data; exports keep the rules about the networks they hold, combining files keeps them all, and GraphML — which has no place for rules — gets the networks already merged. When files are combined, two receivers' records of one call on two merged networks fold into one call (**2 RX**), as they would on one network.

### A.7 Audio

Decoded voice arrives per stream and per slot (DMR's two slots are the left and right channels of dsd-fme's audio). It goes to the open call on that stream and slot. Audio heard just before a call is decoded (up to 1 s) is held and put at the start of the call. Data-only calls never get audio. While only one DMR slot carries voice, dsd-fme copies that voice to both channels. The copy goes to the slot that last showed voice (a voice frame or voice burst), not to the slot of the last line. On a Capacity Plus or trunked channel the other slot sends control bursts nonstop, and following those would cut holes in the call. A copy that lands on a data call on the other slot is given back to the voice call. Encrypted calls are not recorded (unless the session has the key).

---

# Appendix B: Per-protocol notes

| Protocol | Strong identity | Weak identity | Sites | Ids (radio / target) | Slots |
|---|---|---|---|---|---|
| **DMR** | Network id (Tier III, Capacity Max; with network type) | Color code (0–15) | Site id | Radio id → talkgroup or radio | 2 (TDMA) |
| **P25** | WACN + System ID (or System ID alone) | NAC (12-bit) | RFSS + Site | Unit id → talkgroup or unit | Phase 2: 2 |
| **NXDN** | System code | RAN | Site code / location id | Unit id → group or unit | — |
| **TETRA** | MCC + MNC | Colour code | Location area (LA) | SSI → called SSI (group or individual) | — |
| **dPMR** | — | Channel code | — | Radio id → talkgroup | — |
| **D-STAR** | — | Repeater callsign (RPT1) | — | Callsign → destination callsign | — |
| **YSF** | — | Downlink id | — | Callsign → destination | — |
| **EDACS / ProVoice** | System id | — | — | Source → group (or individual) | — |
| **X2-TDMA** | — | — | — | Radio id → talkgroup | as decoded |

**DMR**

- Tier II conventional and repeaters usually give only a **color code**. Send the frequency so each repeater becomes a channel network; otherwise each stream is its own network.
- Tier III trunking and Capacity Max give a **network id** (strong). Capacity Plus reports its site and type (`network_type=cap+`, `site_id`) as identifiers of the color-code network.
- The two timeslots are separate calls; a sync line's slot is carried onto the call lines that follow it when they don't name a slot.
- Capacity Plus **channel-status rosters** (`Bank One … Private or Data Call(s) - LSN …`) list traffic on the site's other channels and are not calls (Appendix A.3).
- SMS: a data preamble, data header and the decoded text become **one** data call carrying the text. Talker aliases are attached to the call and the radio.
- Other data calls are labelled from their headers: a **Response Packet** is an ACK, a **Confirmed / Unconfirmed Delivery** header a data packet, and the Motorola (MNIS) service header that follows names the service (ARS, LRRP, TMS, or its number). Only ARS is decoded; the other services are binary and are no longer scanned for printable bytes (which used to show them as nonsense text like " =&0b" or "/A/"). A data preamble with nothing after it on the channel is *data announced only*.
- Position reports (DMR LRRP / GPS, P25 Motorola LCW GPS) are put on the call and kept as the sending radio's last position.

**P25**

- A control channel broadcasts **WACN / System ID / RFSS / Site**. A voice or conventional channel may only show the **NAC**. Because a NAC is 12 bits (more distinctive than a color code), a NAC-only stream is attached to the known WACN/SysID network with that NAC — when exactly one has it.
- A control-channel grant and the voice channel carrying it are the same call: they are folded into one (**2 RX**).
- Encryption is taken from the algorithm id (anything other than clear / 0x80).

**NXDN**

- **System code** (from control channel broadcasts) is strong; the **RAN** alone is weak. Site code and location id become sites.

**TETRA**

- **MCC + MNC** (from the cell's broadcasts) identify the network; **location area** is the site. The calling SSI is the radio; the called SSI (a group or an individual) is the target — reported by the tetmon decoder path; the tetra-kit path reports only the caller, so its calls show no target and aren't counted as associations. Encryption comes from the decoder's encryption flags.

**dPMR**

- Only a **channel code** — a weak id, so send the frequency to join streams.

**D-STAR and YSF**

- Ids are **callsigns**, not numbers: the source callsign is the radio, the destination (often `CQCQCQ` = everyone) is the "talkgroup".
- The repeater callsign (D-STAR RPT1) or downlink id (YSF) is the network's weak identity, believed at once. Radio text / messages are attached to the call. A simplex D-STAR call (`RPT 1: DIRECT`) has no repeater, so its network is just the channel.
- A D-STAR call picked up after its header shows a blank source (`SRC: ... INTERRUPTED`). It's still a call to its destination: it counts for that "talkgroup" and the channel, with no radio. dsd-fme builds that log frame payloads print the header on every voice frame (`AMBE ... DST:`), and those are read too.

**EDACS / ProVoice**

- The **system id** is strong. The caller is the radio and the group (or, for an individual call, the callee) the target; channel (LCN), AFS and LID numbers are kept as identifiers. This parsing follows dsd-fme's EDACS / ProVoice output formats but hasn't yet been checked against a live signal.

**X2-TDMA**

- No identity is decoded; networks are `Unidentified`, by channel when the frequency is known.

---

# Appendix C: Glossary

| Term | Meaning |
|---|---|
| **Stream** | One decode session: one receiver channel, one protocol. |
| **Network** | A radio system as the explorer identifies it (Appendix A.2). |
| **Merge (networks)** | Showing two or more networks as one — e.g. one repeater heard at two frequency offsets. Shared by everyone on the server and kept in exports; **Unmerge** undoes it (Merging networks). |
| **Chip** | A small rounded, clickable label on the page, like a tag. In the **Filters**, each network chip names one network (with its colour swatch and call count), and each talkgroup, radio or search in force has one. A highlighted chip is picked, a dashed struck-through one excluded; click it to drop it from the filter. |
| **Strong / weak / channel identity** | How a network was identified: a unique system id / only a short shared code / a short code on a known frequency. |
| **Talkgroup (TG)** | A group address many radios listen to. |
| **Private call** | Radio-to-radio (unit-to-unit, individual) call; ⇄ in the To column. |
| **Slot** | A TDMA timeslot: DMR has 2 per channel, TETRA 4. Each carries its own call. |
| **2 RX** | One call heard by two receivers / streams, listed once. |
| **Talker alias** | A name the radio sends with its id (DMR, P25). |
| **Color code / NAC / RAN** | Short codes that keep co-channel systems apart (DMR / P25 / NXDN). Not unique. |
| **WACN / System ID** | P25's wide-area and system identifiers. |
| **LSN** | Logical slot number: a channel and slot on a Capacity Plus site. |
| **Export** | The explorer's gathered data saved as a file (re-openable). |
| **Recording** | The explorer's raw input saved for exact replay (troubleshooting). |

## Appendix D: The `calls.csv` columns

⭳ **Audio (.zip)** (§10) packs a `calls.csv` beside the WAV files — one row per call saved, in the order the Calls list showed them (so filter, search or sort first). It has these 15 columns:

| # | Column | Meaning |
|---|---|---|
| 1 | `file` | The call's WAV filename inside the same zip. |
| 2 | `start_utc` | Call start, UTC — `2026-10-02 17:29:31Z`. |
| 3 | `duration_s` | Call length in seconds (first to last voice), one decimal. |
| 4 | `audio_s` | Length of the recorded audio in seconds, one decimal (≤ `duration_s`; silence isn't recorded). |
| 5 | `network` | The network's label (as shown in the explorer). |
| 6 | `mhz` | Channel frequency in MHz; blank if unknown. |
| 7 | `slot` | TDMA timeslot; blank if none. |
| 8 | `from` | Source radio id. |
| 9 | `alias` | Source talker alias, if any. |
| 10 | `to` | Talkgroup id for a group call, or the other radio's id for a private call (see `private`). |
| 11 | `private` | `yes` for a radio-to-radio (private/unit) call, else blank. |
| 12 | `emergency` | `yes` if the call carried an emergency flag, else blank. |
| 13 | `encrypted` | `yes` if the call was encrypted, else blank. |
| 14 | `transcript` | Speech-to-text of the call, if it was transcribed (§11); else blank. |
| 15 | `transcript_model` | The speech model that produced the transcript; else blank. |

Format details:

- **UTF-8 with a BOM** and **CRLF** (`\r\n`) line endings, so it opens cleanly in Excel and Google Sheets.
- A field containing a comma, quote or newline is wrapped in double quotes, with any inner quote doubled (standard CSV quoting). A field that begins with `=`, `+`, `-` or `@` is prefixed with a single quote (`'`) so a spreadsheet doesn't treat it as a formula.
- Only **finished calls that still have audio on the server** are included. The zip (and so the CSV) stops at 25 MB; if it fills, the rows past that point are left out and a message says how many.
- Times are UTC and absolute, so a CSV from a file view matches that export's own clock, not yours.